

Analizador léxico

Documento de diseño

Materia	Compiladores · UNAM, Facultad de Ingeniería · Grupo 5
Paquete	unam.fi.compilers.g5.02 (equipo 2)
Fecha de entrega	6 de octubre de 2026
Rol	Diseñador (define qué hace el lexer y cómo se organiza)

1. Objetivo y alcance

Este documento define el analizador léxico (lexer) del equipo 02. El lexer lee el código, separa los lexemas, indica de qué tipo es cada uno y al final reporta las categorías y el total de tokens.

- Hace: lee una cadena o un archivo, reconoce 40 tipos de token, ignora espacios y comentarios y reporta los errores léxicos con su línea y columna.
- No hace análisis sintáctico. La CFG con left factoring de la sección 7 es solo el requisito teórico del reporte.
- Restricciones del profesor: no se puede usar FLEX. Hacerlo a mano, solo con lo visto en clase, da puntos extra. Usar librerías de análisis léxico también es válido, pero sin puntos extra.
- Implementación: scanner escrito a mano en Python. Cada estado del DFA es una función, y el código no usa FLEX, librerías de análisis léxico ni expresiones regulares. Las expresiones de la sección 2 solo describen cada token.
- Alcance: el lenguaje es pequeño y se puede ampliar. Llevarlo a la gramática completa de Solidity (más de 80 palabras clave) queda para después y no es parte de esta entrega.

Decisiones de diseño

- `print` es palabra reservada porque lo pide el profesor. `printf` también es keyword porque aparece en el ejemplo. `printed` y `printx` son identificadores.
- El lenguaje distingue mayúsculas de minúsculas: `Print` es un identificador.
- Los comentarios (`#` hasta el fin de la línea), los espacios, las tabulaciones y los saltos de línea se descartan y no cuentan como tokens.
- Los números no llevan signo: `-` siempre es MINUS, así que `10-3` son tres tokens.

2. Catálogo de tokens (40)

#	Token	Categoría	Lexemas / expresión regular	Ejemplo	Qué no es
1	START	keyword	Start	Start	start
2	IF	keyword	if	if	ifx
3	THEN	keyword	then	then	thenx
4	ELSE	keyword	else	else	elsewhere
5	WHILE	keyword	while	while	whiles
6	FOR	keyword	for	for	format
7	FROM	keyword	from	from	fromage
8	TO	keyword	to	to	total
9	PRINT	keyword	print	print	printf, printed
10	PRINTF	keyword	printf	printf	print
11	READ	keyword	read	read	ready
12	INT	keyword	int	int	integer
13	REAL	keyword	real	real	realm

#	Token	Categoría	Lexemas / expresión regular	Ejemplo	Qué no es
14	STRING_TYPE	keyword	string	string	strings
15	BOOL_TYPE	keyword	bool	bool	boolean
16	AND	operator	and	and	android
17	OR	operator	or	or	order
18	NOT	operator	not	not	note
19	BOOL_LITERAL	constant	(True False)	True	true
20	EQ	operator	==	==	=
21	NEQ	operator	!=	!=	! suelto (error)
22	LEQ	operator	<=	<=	<
23	GEQ	operator	>=	>=	>
24	ASSIGN	operator	=	=	==
25	LT	operator	<	<	<=
26	GT	operator	>	>	>=
27	PLUS	operator	\+	+	
28	MINUS	operator	-	-	signo de número
29	MULT	operator	*	*	
30	DIV	operator	/	/	
31	REAL_NUMBER	constant	\d+\.\d+	3.14	3. (error)
32	INTEGER	constant	\d+	10	-10 (MINUS + INTEGER)
33	STRING_LITERAL	constant	"[^"]*"	"This is an example"	cadena sin cerrar (error)
34	ID	identifier	[a-zA-Z_][a-zA-Z0-9_]*	a, x_1	1abc (se lee 1 + abc)
35	L_PAREN	punctuation	\(	(	
36	R_PAREN	punctuation	\)	)	
37	L_BRACE	punctuation	\{	{	
38	R_BRACE	punctuation	\}	}	
39	SEMICOLON	punctuation	;	;	
40	COMMA	punctuation	,	,	

Se ignoran (no son tokens): `#[^\n]*` (comentario) y `\s+` (espacios, tabulaciones y saltos de línea).
Delimitación de palabras: en vez de `\b`, el DFA lee en q1 el lexema completo (letras, dígitos y `_`) y después lo busca en la tabla de palabras reservadas. La palabra termina en el primer carácter que no sea letra, dígito ni `_`. Por eso `ifx` es ID e `if` es keyword.

Orden de las reglas: para un lexema con letras se revisa primero si es booleano, luego si es palabra reservada (incluye `and`, `or` y `not`) y si no, es ID. Los operadores de dos caracteres se prueban antes que los de uno, y `REAL_NUMBER` antes que `INTEGER`.

2.1 Correspondencia con las categorías del diseño arquitectónico

El diseño arquitectónico usa 7 categorías. El Writer las agrupa en 5 para el reporte:

Categoría conceptual	Categoría del reporte	Tokens
Keyword	keyword	los 15 keywords (START ... BOOL_TYPE)
Identifier	identifier	ID
Operator	operator	aritméticos, asignación/comparación y lógicos (AND, OR, NOT)
Constants	constant	INTEGER, REAL_NUMBER, BOOL_LITERAL
Literals	constant	STRING_LITERAL
Punctuators	punctuation	los 6 signos de puntuación
Special_Characters (\$ @ #)	— (sin token)	\$ y @ son carácter ilegal (error léxico); # inicia un comentario que se descarta

3. Contrato de salida (formato implementado)

El Writer genera un reporte de texto que se imprime en consola y se puede guardar en un archivo. Tiene tres partes: el resumen por categoría con el total, el detalle de cada token y los errores léxicos. Ejemplo con `int a = 10;;`:

- ANALIZADOR LÉXICO -

keyword identifier operator constant punctuation

Total de tokens: 5

- DETALLE DE TOKENS -

No.	Tipo	Categoría	Lexema	Línea	Columna
1	INT	keyword	'int'	1	1
2	ID	identifier	'a'	1	5
3	ASSIGN	operator	'='	1	7
4	INTEGER	constant	'10'	1	9
5	SEMICOLON	punctuation	';'	1	11

- ERRORES LÉXICOS -

Sin errores léxicos.

Las categorías son keyword, identifier, operator, constant y punctuation. Cada tipo interno (INT, ID, ASSIGN, etc.) pertenece a una de ellas. El lexema se muestra entre comillas simples y la línea y la columna empiezan en 1.

Recuperación de errores (Panic Mode): cuando aparece un carácter ilegal o un lexema mal formado, el análisis no se detiene. El error se guarda con su línea y columna, se descarta el lexema y se sigue desde la siguiente posición válida. Así se pueden detectar varios errores en una sola ejecución. Los errores no cuentan en el total de tokens y se imprimen como Línea L, columna C: mensaje.

Entrada	Mensaje del reporte
@	Carácter ilegal: '@'
3.	Número mal formado: '3.'
! suelto	Operador inválido: '!'
"abc	Literal de cadena sin cerrar: '"abc' (se lee hasta el fin de la entrada)

`1abc` no es un error léxico: el lexer lo lee como INTEGER (1) seguido de ID (`abc`). Detectar ese caso le toca a una fase posterior.

Ejemplo: `int a = 3.; b = ! @ $;` da 7 tokens y 4 errores, en las columnas 9, 17, 19 y 21:

Línea 1, columna 9: Número mal formado: '3.'
Línea 1, columna 17: Operador inválido: '!'
Línea 1, columna 19: Carácter ilegal: '@'
Línea 1, columna 21: Carácter ilegal: '\$'

4. Arquitectura

4.1 Estructura del paquete

```
develop/
├── examples/
│   └── ejemplo.txt
├── unam/fi/compilers/g5/02/
│   ├── main.py           ← orquesta Reader → Scanner → Writer y CLI
│   ├── io/
│   │   ├── reader.py     ← fuente desde cadena o archivo
│   │   └── writer.py     ← reporte (resumen, total, tabla, errores)
│   ├── lexer/
│   │   ├── dfa.py        ← funciones de estado del DFA
│   │   └── scanner.py    ← puntero, línea, columna, tokens y errores
│   ├── models/
│   │   └── token.py      ← Token inmutable
│   ├── symbol_table/
│   │   └── symtab.py     ← keywords, booleanos e identificadores
│   └── tests/           ← pruebas unitarias y de integración
```

4.2 Arquitectura en capas

main.py: CLI (argparse) que orquesta el flujo
Reader: obtiene la fuente (cadena o archivo)
Scanner: recorre carácter por carácter y coordina los estados
DFA (dfa.py) + SymbolTable (keywords / identificadores)
Writer: resumen por categoría + total + tabla + errores

Flujo: main.py pasa la fuente al Scanner, que usa las funciones del DFA y la tabla de símbolos; la lista de tokens y errores llega al Writer.

4.3 Flujo de datos

texto → caracteres → lexemas (DFA) → tokens (tipo, lexema, línea, col) → reporte (resumen + tabla + total + errores)

4.4 Módulos y clases

Módulo	Responsabilidad
main.py	punto de entrada; CLI con --string, --file, --output
io/reader.py	obtiene la fuente desde cadena o archivo (UTF-8)
io/writer.py	construye y emite el reporte
lexer/dfa.py	funciones de estado: identificador, número, cadena, operadores, puntuación
lexer/scanner.py	puntero, línea y columna; crea tokens; registra errores (Panic Mode)
models/token.py	Token inmutable: tipo, lexema, línea, columna
symbol_table/symtab.py	palabras reservadas, booleanos e identificadores (lookup / insert)
tests/	pruebas unitarias y de integración

4.5 Tabla de símbolos (inserción temprana)

Al formar un lexema con letras o dígitos, el Scanner consulta la tabla de símbolos en este orden: (1) literal booleano (True, False) y se emite `BOOL_LITERAL`; (2) palabra reservada y se emite su token de keyword; (3) cualquier otro lexema se inserta como identificador y se emite `ID`. El Scanner usa tres operaciones: `is_boolean_literal`, `lookup_keyword` e `insert`.

Campo	Uso en la fase léxica
name	nombre del identificador
type	reservado para la fase semántica
size	reservado
dimension	reservado
declaration_line	se registra la primera línea donde aparece
usage_line	a completar en fases posteriores
address	a completar en fases posteriores

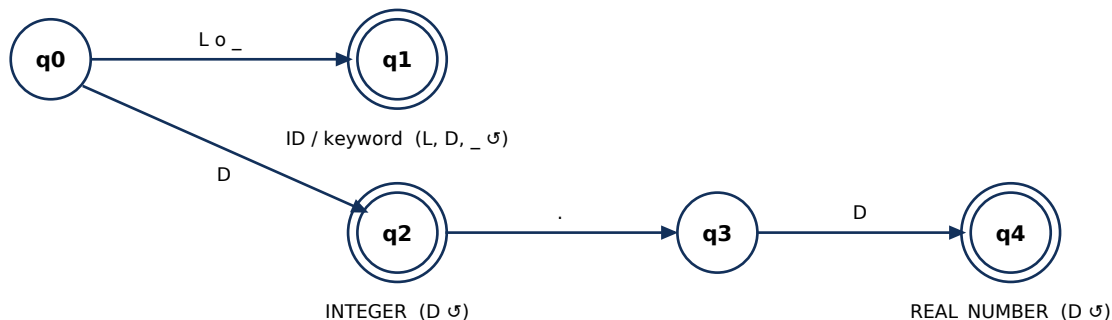
Ejemplo tras analizar `int a = 10;; a` con `declaration_line = 1` y el resto de los campos pendientes (TBD).

5. Autómata (DFA) y tabla de transiciones

Alfabeto: L letra (el código usa `isalpha()`, por lo que también acepta letras acentuadas), D dígito, _ WS espacio/tab/salto, # comentario, y los símbolos de operadores y puntuación.

Nota: el equipo programa cada estado como una función en `dfa.py` (State Pattern) y no como una matriz de transiciones. La tabla de la sección 5.2 describe el comportamiento de esas funciones y sirve de referencia para las pruebas.

5.1 DFA de identificador, entero y real



Keyword `print`: el DFA lee el lexema completo en `q1` y al terminar lo busca en la tabla de palabras reservadas (`Start`, `if`, `then`, `else`, `while`, `for`, `from`, `to`, `printf`, `print`, `read`, `int`, `real`, `string`, `bool`, `and`, `or`, `not`, `True`, `False`). Si está, emite su token de keyword; si no, emite `ID`. Por eso `printf` y `printed` no se confunden con `print`.

5.2 Tabla de transiciones

Estado	Símbolo	Siguiente	Acción
q0	L, _	q1	—
q0	D	q2	—
q0	"	q5	—
q0	#	q11	—
q0	= / ! / < / >	q7 / q8 / q9 / q10	—
q0	+ - * / () { } ; ,	q12	emitir token correspondiente
q0	WS	q0	ignorar

Estado	Símbolo	Siguiente	Acción
q0	otro	—	error léxico (carácter no reconocido)
q1	L, D, _	q1	—
q1	otro	q12	emitir keyword o ID (tabla de reservadas)
q2	D	q2	—
q2	.	q3	—
q2	otro (incluye letras)	q12	emitir INTEGER (1abc → 1 + abc)
q3	D	q4	—
q3	otro	—	error léxico (3.)
q4	D	q4	—
q4	otro	q12	emitir REAL_NUMBER
q5	"	q6	—
q5	otro (incluye saltos de línea)	q5	—
q5	fin de entrada	—	error léxico (cadena sin cerrar; el lexema llega hasta el fin)
q6	cualquiera	q12	emitir STRING_LITERAL
q7	=	q12	emitir EQ
q7	otro	q12	emitir ASSIGN
q8	=	q12	emitir NEQ
q8	otro	—	error léxico (! suelto)
q9	=	q12	emitir LEQ
q9	otro	q12	emitir LT
q10	=	q12	emitir GEQ
q10	otro	q12	emitir GT
q11	\n	q0	ignorar comentario
q11	otro	q11	—
q12	—	q0	reiniciar para el siguiente token

Máxima longitud (maximal munch): el autómata sigue por la transición válida más larga antes de aceptar (== antes que =, 3.14 antes que 3). Los caracteres que cuentan como "otro" no se consumen al aceptar.

6. Interfaz y salida (propuesta de UI, baja fidelidad)

El programa se ejecuta por línea de comandos (`python -m unam.fi.compilers.g5.02.main --string ... | --file ...`). El wireframe de abajo es una propuesta opcional de interfaz gráfica; muestra la entrada y el formato del resultado.

Analizador léxico - Compiladores G5

Entrada:

```
printf("This is an example");
int a = 10;
```

[Subir archivo]

[Cargar ejemplos]

[ANALIZAR]

Resultado:

keyword punctuation constant punctuation ...

Total de tokens: 10

No.	Tipo	Categoría	Lexema	Línea	Columna
1	PRINTF	keyword	'printf'	1	1
...	...	...	...	...	...

Errores léxicos: Sin errores léxicos.

Estilo: sencillo, con los colores de la FI (azul y dorado) o un tema oscuro, y tipografía monoespaciada para el código y los lexemas.

7. CFG y left factoring (requisito teórico)

La CFG usa tokens como terminales, no caracteres. Los terminales son los que produce el lexer (`int`, `id`, `=`, `num`, `;`).

Ejemplo 1: `int a = 10;`

Gramática inicial (factor común `int id` a la izquierda):

```
Stmt → int id = num ;
      | int id ;
```

Después de left factoring:

```
Stmt → int id Rest
Rest → = num ;
      | ;
```

Por qué se factoriza: con dos producciones que empiezan igual, un parser predictivo con un símbolo de lookahead no sabe cuál elegir al ver `int`. Al factorizar, la decisión se pospone hasta ver `=` o `;`.

Ejemplo 2: `printf("This is an example");`

```
Stmt → printf ( string ) ;
      | printf ( string , ArgList ) ;
```

Después de left factoring:

```
Stmt      → printf ( string PrintRest
PrintRest → ) ;
          | , ArgList ) ;
ArgList   → id ArgList'
ArgList'  → , id ArgList' | ε
```

Relación con el lexer

Terminal de la CFG	Token del lexer
<code>int</code>	<code>INT</code>
<code>id</code>	<code>ID</code>
<code>=</code>	<code>ASSIGN</code>

Terminal de la CFG	Token del lexer
num	INTEGER / REAL_NUMBER
string	STRING_LITERAL
; () ,	SEMICOLON L_PAREN R_PAREN COMMA

8. Casos de prueba

#	Entrada	Salida esperada	Total
1	print	keyword	1
2	printf("This is an example");	keyword punctuation constant punctuation punctuation	5
3	int a = 10;	keyword identifier operator constant punctuation	5
4	int a=10;	igual que el anterior (los espacios no cuentan)	5
5	printf("This is an example"); int a = 10;	los 10 tokens de los casos 2 y 3	10
6	Print	identifier (case-sensitive)	1
7	printed	identifier	1
8	ifx	identifier	1
9	10-3	constant operator constant	3
10	"a" + "b"	constant operator constant	3
11	a <= b	identifier operator identifier	3
12	x == 3.14	identifier operator constant	3
13	a = 1; # comentario	identifier operator constant punctuation	4
14	@	ERROR: carácter no reconocido	0
15	"abc	ERROR: cadena sin cerrar	0
16	3.	ERROR: real incompleto	0
17	1abc	constant identifier (1 + abc)	2
18	!	ERROR: ! sin =	0
19	int a = 3.; b = ! @ \$;	7 tokens y 4 errores (3., !, @, \$)	7
20	(entrada vacía)	sin tokens	0

9. Relación con el Diseño Arquitectónico v2 (control de cambios)

El documento «Diseño Arquitectónico: Analizador Léxico v2» fue el diseño inicial que se le dio al developer antes de programar. Este documento describe la versión final, que coincide con el código y con el enunciado del profesor. Si hay diferencias, vale lo que dice este documento, porque corresponde a lo que se probó.

Tema	Diseño v2 (inicial)	Versión final / implementación
Cantidad de tokens	Mínimo para los ejemplos (printf, int, operadores básicos)	40 tipos de token (requisito del profesor)
Categorías	7 conceptuales (Keyword, Identifier, Operator, Constants, Literals, Punctuators, Special_Characters)	5 visibles en el reporte; correspondencia en la sección 2.1
Carácter #	Special_Characters, tratado como error	Inicia un comentario hasta fin de línea; no genera token
Números	Solo enteros ([0-9]+)	INTEGER y REAL_NUMBER; el signo es MINUS
Cadenas	^".*"\$	"[^"]*": termina en la primera comilla de cierre
Salida	Formato minimalista en consola y archivo	Reporte completo (resumen, total, detalle y errores) en consola y, con --output, en archivo
CFG	Gramática general con categorías como terminales	Dos ejemplos con tokens concretos (sección 7); el reporte final debe usar una sola
Solidity	Extensión futura	Igual: fuera del alcance de esta entrega
Estructura, State Pattern, Panic Mode, tabla de símbolos, lectura completa	Definidos en v2	Coinciden con la implementación